SECURITY POLICY

Power Glove Vision Security Policy

Reporting, trust boundaries, network exposure, shutdown, and release integrity.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Supported code

Security fixes will be available on the current `main` branch and included in the next tagged release. Older snapshots may lack pairing, network, dependency, or shutdown protections and should be upgraded before troubleshooting them.

Reporting a vulnerability

Please do not publish credentials, pairing codes, tokens, private device configuration, or working exploit instructions in a public issue.

Use the repository's **Security** tab to submit a private vulnerability report when private vulnerability reporting is available. Include:

- the affected commit or release;
- the UNO Q, RetroPie, browser, and network environment involved;
- concise reproduction steps and the observed result;
- the security boundary that was crossed;
- logs or screenshots after removing tokens, passwords, pairing codes, local addresses, and unrelated personal information.

If private reporting is unavailable, open a minimal public issue asking the maintainer to establish a private channel. Do not include exploit details or secrets in that issue. This project does not currently offer a bug bounty or a guaranteed response time.

Ordinary controller mapping, camera compatibility, and game-profile problems may use normal public issues after logs are sanitized.

Security model

PowerGlove Vision is designed for a trusted home or workshop network. The UNO Q performs hand tracking and sends virtual-controller state to RetroPie. RetroPie sends per-game profile changes back to the UNO Q. Neither device should be treated as an Internet-facing service.

The main protected assets are:

- the shared controller token;
- the UNO Q and RetroPie operating systems;
- the privileged `/dev/uinput` receiver;
- the physical pairing display and single-use PIN;
- the fixed-purpose UNO Q shutdown helper;
- the integrity of the App Lab installation ZIP, MediaPipe wheel, downloaded model, and Arduino dependencies.

The project does not attempt to protect a device after an attacker obtains root access, physical storage access, or control of the trusted local network and both paired hosts.

Pairing boundaries

The UNO Q and RetroPie share one random token of at least 16 characters. The active token belongs only in the UNO Q's private `data/device.json` and RetroPie's `/etc/powerglove/token`. It must not be committed, placed in a shell argument, stored in `launcher.json`, or included in a screenshot or log.

The preferred setup path uses authenticated SSH and verifies the remote host key after first trust. Password input is passed through the SSH channel and is not placed on the process command line.

The alternative pairing path uses short-lived TLS, a certificate identity, and a physical single-use PIN displayed on the UNO Q matrix. The browser or client must compare the certificate identity and supply the physical PIN before the token is released. This is a local certificate-pinning ceremony, not validation by a public certificate authority.

Pairing sessions have bounded handshakes and deadlines. Reusing a PIN, removing the physical display requirement, accepting pairing credentials over ordinary HTTP, or extending the listener indefinitely weakens the intended boundary and requires explicit security review.

Network exposure

Port	Protocol	Direction	Boundary
`55355`	UDP	UNO Q to RetroPie	Authenticated virtual-controller packets
`55356`	UDP	RetroPie to UNO Q	HMAC-authenticated profile commands and acknowledgements
`55357`	TCP/TLS	Pairing client to temporary server	Short-lived code-pairing exchange only
`8088`	HTTP	Browser to UNO Q	Local dashboard, public Help guides, diagnostics, and ordinary controls; no pairing credentials accepted
`8443`	HTTPS	Browser to UNO Q	Protected setup and pairing operations

Keep these ports on a trusted LAN. Do not configure router port forwarding, public reverse proxies, cloud tunnels, or Internet firewall exceptions for them. Guest Wi-Fi and untrusted shared networks are inappropriate unless the devices are isolated by firewall rules or a dedicated VLAN.

Controller and profile UDP traffic is authenticated but not encrypted. Anyone with access to the local network can observe packet timing and size even when they cannot create accepted input without the token. The dashboard can expose camera imagery and operational status to clients that can reach it, so network access to port `8088` is itself sensitive.

The Help library serves a fixed list of public Markdown guides and images from the installed application. It must not expose `data/`, the machine-specific cheat sheet, arbitrary filesystem paths, or pairing credentials. Markdown HTML is not executed, unsafe link schemes are rejected, and image requests are confined below `docs/images`.

The dynamic **This cabinet** page accepts only a validated hostname or IP from the browser `Host` header and combines it with `public_config()`. It may show local network addresses, ports, profile selection, camera selection, and whether pairing is configured, but it must never return the token value, passwords, private files, or arbitrary Host-header content.

Shutdown permissions

The web process does not receive general `sudo` permission. A root-owned systemd path unit watches one fixed file in the application data directory. A matching oneshot service deletes that file and requests a non-blocking Linux poweroff.

The dashboard route requires an explicit confirmation header and only creates the fixed request when the host installer has placed the private `.shutdown-enabled` marker. These checks reduce accidents and prevent command substitution; they do not make the dashboard safe for public network exposure. Anyone able to use the reachable dashboard may still cause a denial of service by shutting down the UNO Q.

Keep the path and service units root-owned and mode `0644`. Do not replace the fixed `ExecStart` commands with user input, a shell string, or an arbitrary command runner. Remove or disable both units if remote shutdown is not wanted.

Dependency and release integrity

- The App Lab installation ZIP is generated and verified; it is not maintained as a changing source-controlled binary.
- The custom MediaPipe wheel's provenance and checksum are recorded in `THIRD_PARTY_COMPONENTS.md`.
- Google's Hand Landmarker model downloads from its pinned source and must match the expected SHA-256 digest before atomic installation.
- Arduino library versions are pinned in `sketch/sketch.yaml`.
- GitHub Actions rebuilds and inspects documentation and the App Lab installation ZIP on every pull request and push to `main`.

Changing a download URL, checksum, dependency source, pairing primitive, network binding, file permission, or privileged service requires focused review and corresponding tests and documentation.